Documentación de Arquitectura y Diseño: Agente Inteligente Scooby-Doo

1. Descripción de lo que hay que programar

El juego se divide en siete módulos lógicos, cada uno con una sola responsabilidad, y se comunican siempre hacia abajo, es decir, el arranque conoce a la sesión, la sesión conoce a los niveles, los niveles conocen al mundo, y el mundo conoce a las entidades y a los agentes, pero nunca al revés.

1. **Módulo `nucleo`:** Guarda lo que usa todo el proyecto, `VectorDosDimensiones` y `RectanguloDeColision` para posiciones y colisiones, `MedidorDeRecurso Acumulable` para las barras de tiempo, provisiones y estamina, `Generador Aleatorio` para el azar controlado con semilla, la jerarquía `ExcepcionJuego Base` con sus cuatro hijas, `ExcepcionRecursoNoEncontrado`, `ExcepcionMapa Invalido`, `ExcepcionConfiguracionIncorrecta` y `ExcepcionEstadoInvalido`, y los enums y structs de datos como `EstimuloSensorialPercibido`, `DecisionDe Comportamiento`, `AlertaDeVigilancia` y `EventoDelJuego`.

2. **Módulo `fisicas`:** Tiene la clase abstracta `ModeloDeFisicaBase` y tres hijas, `ModeloFisicoPenduloColgante` para la lámpara, `ModeloFisicoMovimiento Circular` para el vigilante y `ModeloFisicoTiroParabolico` para los cañonazos, todas calculan la posición con una ecuación en función del tiempo, y sus parámetros los entrega `PerfilDeDificultad`.

3. **Módulo `mapa`:** Tiene `MapaDeCartagena` con la cuadrícula, zonas públicas y restringidas, `ZonaDelEscenario`, y puertas, `PuertaBloqueable`, `RutaDePatrulla` para los recorridos, `BuscadorDeRutasEstrella` con A* para buscar caminos, y `CargadorDe Mapa` que lee el mapa desde archivo y lanza `ExcepcionMapaInvalido` si está mal.

4. **Módulo `entidades`:** Parte de `EntidadDelJuegoBase`, de ahí hereda `Personaje ConMovimientoBase`, y de `PersonajeConMovimientoBase` salen `Personaje Scooby` (Scooby) y `PersonajeAutonomoConIA`, que a su vez da `Sospechoso Investigado`, `SoldadoAliadoDelVillano`, `AmigoSecundarioBase` con su hijo `CentinelaVigilanteDelAlmacen`, y `FantasmaDisfrazado`. Por otro lado `Entidad DelJuegoBase` también da `ObjetoRecolectableBase` con `PistaOcultaDelCaso`, `SuministroDeProvisiones`, `ComodinDeTiempoExtra` y `ComodinDeSprintFruta`, más `TrampaParaElVillano`, `LamparaOscilanteEnPared`, `BalaDeCanonMuralla` y `AlmacenCentralDeProvisiones`.

5. **Módulo `inteligencia`:** Tiene `CerebroAgenteAutonomo`, que es el cerebro, y sus cuatro módulos, `ModuloPercepcionSensorial`, `ModuloRazonamientoBase`,

ModuloAccionBase y ModuloMemoriaYAprendizajeBase, más TableroDeAlertas Compartido para que los personajes se avisen, DirectorDePeligro para subir la presión con cada pista, FabricaDeAgentes para armar cada cerebro, y Habilidad DeAmigoBase con las habilidades de Velma, Fred, Shaggy y Daphne.

6. **Módulo** juego**:** Tiene SesionPrincipalDelJuego, que es la fachada hacia la interfaz, RelojDeAltaPrecision, NivelDelJuegoBase con NivelUnoPrimerasPistas y NivelDosElMisterioPrincipal, MundoDeCartagena, EstadoDeLaMision, Perfil DeDificultad, ArchivoDelCasoCriminal y GeneradorDeMisterios, Progreso DeLaCampana, ColaDeEventosDelJuego, SistemaDeColisiones, GeneradorDe Comodines, SistemaDeBalasDeMuralla y RepositorioDeDialogos.

7. **Módulo** inicio**:** Tiene EjecutorDeLaAplicacion, LanzadorDelJuego, ConstructorDeSesion, CargadorDeConfiguracion y VerificadorDeRecursos, que solo se usan al arrancar.

2. Herencias Propias

Existen nueve jerarquías propias, EntidadDelJuegoBase, ModeloDeFisicaBase, ModuloRazonamientoBase, ModuloAccionBase, ModuloMemoriaYAprendizajeBase, HabilidadDeAmigoBase, NivelDelJuegoBase, ObjetoRecolectableBase y Excepcion JuegoBase. Además, los dos QObject, que son RelojDeAltaPrecision y Sesion PrincipalDelJuego, heredan de Qt, y esa herencia no cuenta como herencia propia.

Nota de diseño clave: La más importante para la rúbrica es que el villano hereda del sospechoso en razonamiento, acción y memoria, así se comporta igual por fuera y es más inteligente por dentro, es decir, RazonamientoVillanoEncubierto hereda de Razonamiento SospechosoComun, AccionVillanoAvanzado hereda de AccionSospechoso, y Memoria AprendizajeVillano hereda de MemoriaAprendizajeSospechoso.

3. El Agente Inteligente

Cada personaje autónomo tiene un CerebroAgenteAutonomo, y cada vez que le toca pensar, cosa que ocurre más o menos cada 0.15 segundos, recorre las cuatro fases en orden:

- **Percepción** (ModuloPercepcionSensorial)**:** Revisa primero la distancia, luego el cono de visión y al final la línea de vista, también lee alertas y detecta si Scooby entró a una zona restringida, y entrega un EstimuloSensorialPercibido.

- **Razonamiento** (ModuloRazonamientoBase)**:** Recibe el EstimuloSensorial Percibido y la memoria, y devuelve una DecisionDeComportamiento como patrullar, acercarse, perseguir o buscar, el villano además puede decidir cerrar una puerta o pedir ayuda a un aliado.

- **Acción** (ModuloAccionBase)**:** Convierte la DecisionDeComportamiento en movimiento por ruta o en un efecto, como ralentizar a Scooby, cerrar una puerta o publicar una alerta, y devuelve un ResultadoDeLaAccion.

- **Aprendizaje** (ModuloMemoriaYAprendizajeBase)**:** La memoria guarda el resultado y cambia las decisiones futuras, el villano sube su sospecha cada vez que ve a

Scooby en una zona y a las tres veces actúa, los aliados aprenden por qué puerta entró Scooby y tienen tres patrullas permitidas en esa zona, los sospechosos recuerdan las zonas calientes, y los amigos no repiten diálogos.

4. Flujo Básico de Ejecución

1. Al arrancar, `main` llama a `EjecutorDeLaAplicacion`, que tiene un solo try/catch.
2. Este llama a `LanzadorDelJuego`, que tiene el try/catch grande y se apoya en `CargadorDeConfiguracion`, `VerificadorDeRecursos` y `ConstructorDeSesion`.
3. El último crea la `SesionPrincipalDelJuego`, y esta crea el nivel, `NivelUno PrimerasPistas` o `NivelDosElMisterioPrincipal`, que arma el `MundoDe Cartagena` con el mapa, las entidades y los cerebros.
4. **Game Loop:** En cada cuadro, `RelojDeAltaPrecision` emite `tick(deltaTiempo)`, `SesionPrincipalDelJuego` lo recibe y llama a `actualizarLogicaDelNivel`, el nivel actualiza el `MundoDeCartagena`, o sea a Scooby, a los personajes, que piensan cuando les toca, y a las físicas.
5. Luego corren los sistemas del nivel, `SistemaDeColisiones` convierte los contactos en `EventoDelJuego`, el nivel aplica las reglas, y al final `SesionPrincipalDelJuego` emite señales Qt hacia el HUD.

5. Reglas de Victoria y Derrota

Condiciones de Derrota: Scooby pierde si pisa la trampa armada, si lo captura un sospechoso, el villano o el fantasma, y si el tiempo o las provisiones llegan a cero.
Condiciones de Interacción: Los aliados no capturan, solo ralentizan a Scooby y llaman al sospechoso más cercano por el `TableroDeAlertasCompartido`, así Scooby tiene tiempo de escapar antes de que llegue, y las persecuciones son breves y empiezan cuando está dentro del rango de visión.
Condiciones de Victoria: Scooby gana si el fantasma pisa la trampa armada.

6. Estructura de Carpetas

```
ScoobyCartagena/
|
|-- main.cpp                      <- 5 líneas, solo llama al ejecutor
|-- ScoobyCartagena.pro           <- o CMakeLists.txt
|
|-- inicio/                       <- arranque repartido en 3 módulos
|   |-- ejecutor_de_la_aplicacion.h/.cpp    <- try/catch de último recurso
|   |-- lanzador_del_juego.h/.cpp           <- try/catch grande, orquesta
|   |-- constructor_de_sesion.h/.cpp        <- crea sesión, azar y perfil
|   |-- cargador_de_configuracion.h/.cpp
|   |-- verificador_de_recursos.h/.cpp
|   `-- configuracion_de_la_aplicacion.h    <- solo struct
|
|-- nucleo/                       <- la carpeta extra que te faltaba
```

```
|   |-- tipos/                    <- vector_dos_dimensiones.h/.cpp,
|   |                               rectangulo_de_colision.h,
|   |                               coordenada_de_casilla.h
|   |-- recursos/                 <- medidor_de_recurso_acumulable.h/.cpp
|   |-- azar/                     <- generador_aleatorio.h/.cpp
|   |-- excepciones/              <- excepcion_juego_base.h/.cpp,
|   |                               excepcion_recurso_no_encontrado,
|   |                               excepcion_mapa_invalido,
|   |                               excepcion_configuracion_incorrecta,
|   |                               excepcion_estado_invalido
|   |-- enumeraciones/            <- todos los enums, un .h por familia
|   |-- datos/                    <- estimulo_sensorial_percibido.h,
|   |                               decision_de_comportamiento.h,
|   |                               alerta_de_vigilancia.h,
|   |                               evento_del_juego.h,
|   |                               resultado_de_la_accion.h,
|   |                               estado_de_entrada_teclado.h
|   `-- constantes/               <- constantes_del_juego.h,
|                                   velocidades y límites
|
|-- logica/                       <- módulos fisicas, mapa, entidades, juego
|   |-- sesion/                   <- sesion_principal_del_juego,
|   |                               reloj_de_alta_precision,
|   |                               cola_de_eventos_del_juego,
|   |                               progreso_de_la_campana
|   |-- niveles/                  <- nivel_del_juego_base,
|   |                               nivel_uno_primeras_pistas,
|   |                               nivel_dos_el_misterio_principal,
|   |                               fabrica_de_niveles,
|   |                               estado_de_la_mision
|   |-- dificultad/               <- perfil_de_dificultad
|   |-- casos/                    <- archivo_del_caso_criminal,
|   |                               generador_de_misterios
|   |-- mundo/                    <- mundo_de_cartagena,
|   |                               sistema_de_colisiones
|   |-- mapa/                     <- mapa_de_cartagena,
|   |                               zona_del_escenario.h,
|   |                               puerta_bloqueable.h,
|   |                               ruta_de_patrulla,
|   |                               buscador_de_rutas_estrella,
|   |                               cargador_de_mapa
|   |-- fisicas/                  <- modelo_de_fisica_base,
|   |                               modelo_fisico_pendulo_colgante,
|   |                               modelo_fisico_movimiento_circular,
|   |                               modelo_fisico_tiro_parabolico,
|   |                               parametros_de_fisica.h
|   |-- entidades/
|   |   |-- base/                 <- entidad_del_juego_base,
|   |   |                           personaje_con_movimiento_base
|   |   |-- jugador/              <- personaje_scooby
|   |   |-- autonomos/            <- personaje_autonomo_con_ia,
|   |   |                           sospechoso_investigado,
|   |   |                           soldado_aliado_del_villano,
|   |   |                           fantasma_disfrazado,
|   |   |                           amigo_secundario_base,
|   |   |                           centinela_vigilante_del_almacen
|   |   |-- items/                <- objeto_recolectable_base,
|   |   |                           pista_oculta_del_caso,
```

```
|   |   |                              suministro_de_provisiones,
|   |   |                              comodin_de_tiempo_extra,
|   |   |                              comodin_de_sprint_fruta
|   |   `-- escenario/           <- trampa_para_el_villano,
|   |                              lampara_oscilante_en_pared,
|   |                              bala_de_canon_muralla,
|   |                              almacen_central_de_provisiones
|   |-- sistemas/                <- generador_de_comodines,
|   |                              sistema_de_balas_de_muralla,
|   |                              repositorio_de_dialogos
|   `-- menu/                    <- estado_del_juego.h, reservada para
|                                  menú y selección de dificultad
|
|-- inteligencia/                <- módulo inteligencia
|   |-- agente/                  <- cerebro_agente_autonomo,
|   |                              fabrica_de_agentes
|   |-- percepcion/              <- modulo_percepcion_sensorial,
|   |                              perfil_de_percepcion.h
|   |-- razonamiento/            <- modulo_razonamiento_base,
|   |                              razonamiento_sospechoso_comun,
|   |                              razonamiento_villano_encubierto,
|   |                              razonamiento_soldado_aliado,
|   |                              razonamiento_amigos_secundarios
|   |-- accion/                  <- modulo_accion_base,
|   |                              accion_sospechoso,
|   |                              accion_villano_avanzado,
|   |                              accion_soldado_aliado,
|   |                              accion_amigo_secundario
|   |-- memoria/                 <- modulo_memoria_y_aprendizaje_base,
|   |                              memoria_aprendizaje_sospechoso,
|   |                              memoria_aprendizaje_villano,
|   |                              memoria_aprendizaje_soldado_aliado,
|   |                              memoria_aprendizaje_amigos
|   |-- comunicacion/            <- tablero_de_alertas_compartido,
|   |                              director_de_peligro
|   `-- habilidades_amigos/      <- habilidad_de_amigo_base,
|                                  habilidad_de_informacion,
|                                  habilidad_de_regalo,
|                                  habilidad_de_pista_sutil,
|                                  habilidad_de_refugio
|
|-- auxiliares/                  <- funciones libres, no son clases
|   |-- geometria_aux.h/.cpp     <- clamp, ángulos, distancias
|   |-- validacion_aux.h/.cpp    <- revisar parámetros y lanzar excepciones
|   |-- texto_aux.h/.cpp         <- enum a texto para mensajes y registros
|   `-- ensambladores/           <- ensamblador_del_mundo,
|                                  ayudante_de_aparicion,
|                                  constructor_de_rutas
|
|-- interfaz/                    <- vacía por ahora, Momento III
|   `-- (ventana Qt, vistas, HUD, sonido)
|
|-- estaticos/
|   |-- imagenes/
|   |   |-- sprites/             <- scooby, sospechosos, fantasma, aliados,
|   |   |                          amigos, items, escenario
|   |   |-- fondos/
|   |   |-- tiles/
```

```
|   |   `-- interfaz/
|   |-- mapas/                    <- nivel1.map, nivel2.map
|   |-- sonidos/                  <- musica/, efectos/
|   `-- datos/                    <- dialogos.txt, config.ini,
|                                    dificultades.txt
|
`-- documentacion/                <- momentos I, II y III, diagramas, informe
```

Cada módulo se divide en `.h` y `.cpp`, y los structs y enums quedan solo en `.h`. Todo lo de `auxiliares` son funciones libres dentro de un namespace, por eso no aparecen en el diagrama de clases, y si en algún momento una de ellas crece hasta ser una clase, se agrega al diagrama.

La carpeta extra que sentías que faltaba es `nucleo`. Ahí vive todo lo que usan los demás módulos, y evita que `logica` e `inteligencia` se importen entre sí en círculo.